Voice-LLM Workbench

Part II Progress Presentation

Vedaangh Rungta

University of Cambridge

February 2026

Introduction

Goal: Build a platform to scalably train and deploy speech on top of existing open-source LLM backbones.

Published as a conference paper at ICLR 2025

LLAMA-OMNI: SEAMLESS SPEECH INTERACTION WITH LARGE LANGUAGE MODELS

Qingkai Fang[1], Shaolei Guo[2,3], Yan Zhou[2,*], Zhengrui Ma[1], Shoutao Guo[1], Yang Feng[1,†]

[1]Key Laboratory of Intelligent Information Processing
Institute of Computing Technology, Chinese Academy of Sciences (ICT/CAS)
[2]Key Laboratory of AI Safety, Chinese Academy of Sciences
[3]University of Chinese Academy of Sciences, Beijing, China
{fangqingkai,guoshaolei,zhouyan,mazhengrui,guoshoutao,yang.feng}@ict.ac.cn

ABSTRACT

Models like GPT-4o enable real-time interaction with large language models (LLMs) through speech, significantly enhancing user experience compared to traditional text-based interaction. However, there is still a lack of exploration on how building LLM-based speech interaction models based on open-source LLMs. On top of open-source LLMs, we propose LLAMA-OMNI, a novel end-to-end model architecture designed for low-latency and high-quality speech interaction with LLMs. LLAMA-OMNI integrates a pre-trained speech encoder, a speech adaptor, an LLM, and a streaming speech decoder. It eliminates the need for speech transcription, and can simultaneously generate text and speech responses directly from speech instructions with extremely low latency. We build our model based on the latest Llama-3.1-8B-Instruct model. To align the model with speech interaction scenarios, we construct a dataset named InstructS2S-200K, which includes 200K speech instructions and corresponding speech responses, with a style that better matches the characteristics of speech interaction scenarios. Experimental results show that compared to previous speech-language models, LLAMA-OMNI provides better responses in both content and style, with a response latency as low as 236ms. Additionally, training LLAMA-OMNI takes less than 3 days on just 4 GPUs, paving the way for the efficient development of speech-language models in the future.

1 INTRODUCTION

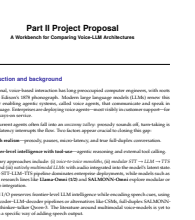
Large language models (LLMs), represented by ChatGPT (OpenAI, 2022), have become powerful general-purpose task solvers, capable of assisting people in daily life through conversational interactions. However, most LLMs currently only support text-based interactions, which limits their interaction speed and bandwidth in scenarios where text input and output are impractical. Recently, GPT-4o (OpenAI, 2024) has made it possible to interact with LLMs through speech, responding to user's instruction with extremely low latency and significantly enhancing the user experience. Therefore, there is a strong demand for high-quality and low-latency speech interaction with open-source models based on LLMs. Therefore, how to achieve low-latency and high-quality speech interaction with LLMs is a pressing challenge that needs to be addressed.

The simplest way to enable speech interaction with LLMs is through a cascaded system based on automatic speech recognition (ASR) and text-to-speech (TTS) models, where the ASR model transcribes the user's speech instruction into text for the LLM to understand, and the TTS model converts LLM responses into speech. However, since the cascaded system sequentially outputs the transcribed text, text response, and speech response, the overall system tends to have higher latency. In contrast, some multimodal speech-language models have been proposed (Zhang et al., 2023; Rubenstein et al.,

*Corresponding Author: Yang Feng.
Code: https://github.com/ictnlp/LLaMA-Omni
Model: https://huggingface.co/ICTNLP/Llama-3.1-8B-Omni
Audio Samples: https://ictnlp.github.io/Llama-omni-demo/

1

arXiv:2409.06666v2 [cs.CL] 1 Mar 2025



Part II Project Proposal

A Workbench for Comparing Voice-LLM Architectures

1 Introduction and background

Conversational, voice-based interaction has long preoccupied computer engineers, with roots traceable to Edison's 1878 phonograph. Modern large language models (LLMs) renew this ambition by enabling agentic systems, called voice agents, that communicate and speak in natural language. Enterprises are deploying voice agents—most visibly in customer support—for scalable, always-on service.

However, current agents often fall into an *anyone talking* protocol: mutually sounds off, turn-taking is brittle, and latency interrupts the flow. Two factors appear crucial to closing this gap:

1. **Speech realism**—prosody, pauses, micro-latency, and true full-duplex conversation.
2. **Frontier-level intelligence with tool use**—agentic reasoning and external tool calling.

Contemporary approaches include: (i) *voice-to-voice* monoliths, (ii) modular ASR → LLM → TTS pipelines, and (iii) *speech-native* LLMs that extend text models with speech understanding and generation. Notably, the ASR–LLM–TTS pipeline dominates enterprise deployments, while models such as GPT-4o and research lines like **Llama-Omni** (S2I) and **SALMONN-Omni** explore modular or native audio integration.

Multimodal LLO preserves frontier-level LLM intelligence while escaping speech-cues, using modular encoder–LLM–decoder pipelines or alternatives like CASA, full-duplex SALMONN-Omni, and thinker–talker Qwen-3. The literature around multimodal voice-models is yet to converge on a specific way of adding speech output.

A persistent problem in the literature is comparability: papers evaluate whole stacks rather than isolating audio-integration design choices. This makes sense as the papers are stacking the releases of voice-LLMs, rather than the architectural changes themselves. However, gains may come from a different base LLM rather than the modality integration itself, and posting integrations across backbones is awkward. We therefore propose a maintainable workbench to combine any open-source text-only LLMs with a voice-integration strategy and evaluate them head-to-head under a standard interface and a common benchmark (**VoiceBench**).

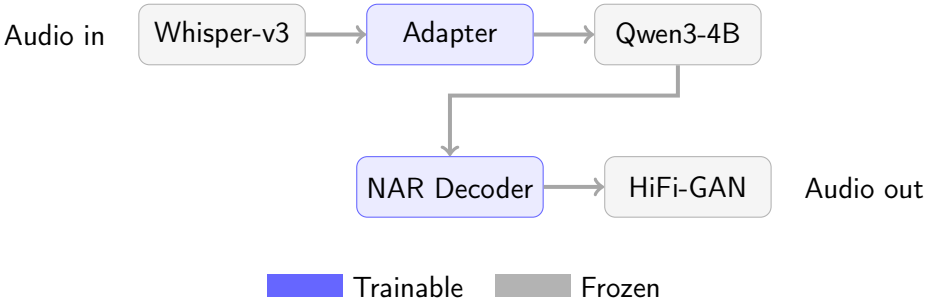
2 Description of work and success criteria

2.1 The Workbench

The workbench will be a Python library that can pair any text-only LLM with a Llama-Omni style encoder–LLM–decoder setup, expose a **uniform streaming API**, and run **VoiceBench** on any given speech-LLM. The primary backbone is **Qwen3-8B** (for feasibility), with targeted

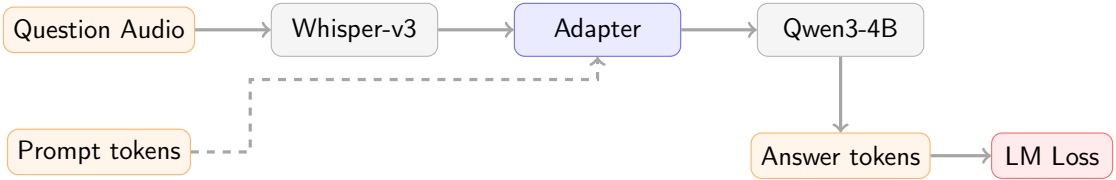
1

Current Work: Architecture



Two-stage training: (1) Train adapter with LM loss (2) Train decoder with CTC loss

Encoder Training (Stage 1)



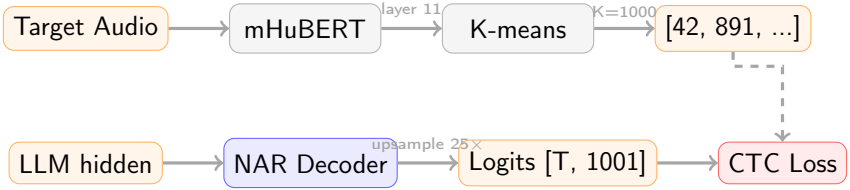
What's trained

- Adapter only (2-layer MLP)
- Whisper and Qwen frozen
- Maps Whisper dim → Qwen dim

Loss

- Standard next-token prediction
- Cross-entropy on answer tokens
- Teacher forcing

Decoder Training (Stage 2)



Unit extraction (offline)

- mHuBERT: self-supervised speech repr.
- K-means quantizes to 1000 clusters
- Deduplicate consecutive repeats

CTC loss

- No alignment labels available
- Sum prob. of *all* valid alignments
- Blank token absorbs timing slack

Why sum all alignments? We don't know which frames map to which units. CTC marginalizes over this uncertainty; any alignment that produces the correct sequence contributes to the loss.

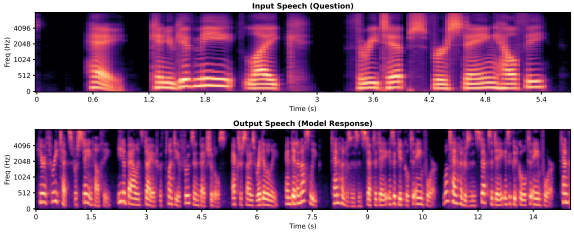
Current Work: Results

Training on H100

Encoder training	7.5 hours
Decoder training	16.5 hours
Encoder val loss	1.60
Decoder val loss	0.74

Output coherent for 7-8 seconds, then degrades.

Decoder undersized: 1024 vs 4096 hidden dim.



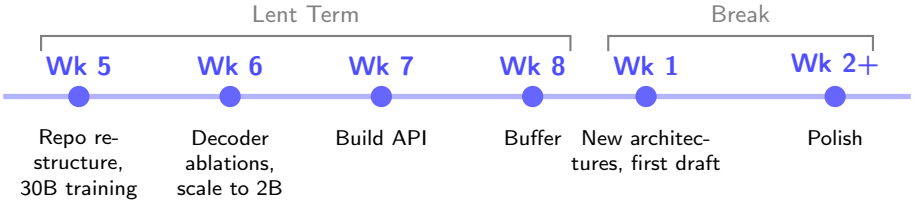
Input (top) vs output (bottom) spectrograms

Architectural Design

Splitting the codebase into **product** and **research**.

Product	Research
Pick voice, dataset, base model Kick off training via API Serve models (serverless) Run VoiceBench evaluations	Run ablations Try new architectures Measure quality
FastAPI, Modal, S3, open-source TTS	SkyPilot, Hydra, WandB

Timeline



12 weeks to deadline

Reflections

- SkyPilot + Hydra + WandB makes ablations a single command
- Writing gets to 80% fast; the last 20% takes just as long
- Original plan targeted ICLR workshop deadlines; submitted different work there instead
- GPU quotas were painful early on, now have A100 clusters, H100s coming
- Still somewhat capital-constrained
- Key decision: how deep to go on infrastructure? Modal gives autoscaling but costs money. Custom is cheaper but more work.